

HELIX

Cybersecurity Operations Platform – Proof of Concept

Blue Team Monitoring · AI-Assisted Analysis · ML Anomaly Detection

Document Type	Software Requirements Specification (SRS)
Project Phase	Proof of Concept (PoC)
Document Version	1.0
Academic Year	2025 – 2026
Author	Yasseene
Status	Final
Classification	Academic — Confidential

HELIX is a modular cybersecurity platform that unifies defensive monitoring, AI-assisted threat analysis, machine learning anomaly detection, and file integrity monitoring. This SRS defines the requirements for the Proof of Concept (PoC) stage, covering authentication, log ingestion, alert visualization, ML scoring, AI assistance, Red Team tool integration, and file integrity monitoring.

Revision History

Version	Date	Author	Description
1.0	May 2026	Yasseene	PoC SRS – Core requirements and scope definition

Contents

Revision History	1
1 Project Presentation	3
1.1 Project Identity	3
1.2 Technology Stack	4
1.3 PoC Scope	4
2 Context and Problem Statement	5
2.1 Industry Context	5
2.2 Core Problem Statement	5
2.3 Limitations of Existing Solutions	5
2.4 HELIX Positioning	5
3 Project Objectives	7
3.1 General Objective	7
3.2 Functional Objectives	7
3.3 Academic Objectives	7
3.4 Non-Functional Objectives	8
4 User Roles and System Actors	9
4.1 Actor Overview	9
4.2 Detailed Actor Descriptions	10
4.2.1 Actor A0 – Authenticated User (Abstract)	10
4.2.2 Actor A1 – Administrator	10
4.2.3 Actor A2 – Blue Team Operator	11
4.2.4 Actor A3 – Red Team Operator	11
4.2.5 Actor A4 – Purple Team Operator	11
4.2.6 Actor A5 – Learner	12
4.2.7 Actor A6 – Visitor (Unauthenticated)	12
4.2.8 Actor A7 – Detection Engine (Secondary System Actor)	13
5 Functional and Non-Functional Requirements	14
5.1 Functional Requirements	14

5.1.1	Module 1 – Authentication and Access Control	14
5.1.2	Module 2 – Dashboard	14
5.1.3	Module 3 – Log Ingestion	15
5.1.4	Module 4 – Alert Visualization	15
5.1.5	Module 5 – ML Anomaly Detection	16
5.1.6	Module 6 – AI Assistant	16
5.1.7	Module 7 – File Integrity Monitoring	16
5.1.8	Module 8 – Red Team Tool	17
5.2	Non-Functional Requirements	17
5.2.1	Performance	17
5.2.2	Security	18
5.2.3	Maintainability	18
5.2.4	Usability	18
5.2.5	Deployability	18

Conclusion		19
-------------------	--	-----------

1. Project Presentation

1.1 Project Identity

Project Name	HELIX
Subtitle	Cybersecurity Operations Platform – Proof of Concept
Application Type	Modular web application with service-oriented components
Domain	Cybersecurity, Machine Learning, Artificial Intelligence
Keywords	SIEM, Blue Team, Red Team, ML Anomaly Detection, AI Assistant
Institution	Tek-Up University, Ariana, Tunisia

HELIX takes its name from the DNA double helix: two strands – offensive (Red) and defensive (Blue) – that together form a more capable whole. The PoC demonstrates the foundational capabilities of this unified platform.

1.2 Technology Stack

Layer	Technology	Role
Frontend	HTML5 / CSS3 / JavaScript	User interface and visualization
Backend / API	PHP	REST API, authentication, business logic
Database	MySQL (XAMPP)	Data persistence and querying
ML Service	Python (scikit-learn)	Isolation Forest anomaly detection
Java Module	Java JDK	SHA-256 hashing and file integrity scanning
AI Integration	OpenAI API	Alert explanation and analysis

1.3 PoC Scope

The PoC validates core platform viability through eight deliverable modules:

1. Authentication with role-based access control (Administrator, Blue Team Operator, Red Team Operator, Purple Team Operator, Learner, Visitor)
2. Centralized security dashboard with key metrics
3. Manual log ingestion and parsing
4. Alert visualization with severity classification
5. ML-based anomaly scoring using Isolation Forest
6. AI-powered assistant for alert explanation
7. Red Team tool integration (network port scanner)
8. File Integrity Monitoring (SHA-256 baseline and verification)

2. Context and Problem Statement

2.1 Industry Context

Global cybercrime costs continue to rise exponentially, yet security education and training lack affordable, integrated environments where students and professionals can practice both defensive and offensive security in a single platform.

2.2 Core Problem Statement

How to design and implement a unified, accessible cybersecurity platform that combines threat monitoring, AI-assisted analysis, and machine learning anomaly detection for academic and training purposes?

2.3 Limitations of Existing Solutions

Solution	Strengths	Limitations	Cost
Splunk	Mature SIEM, rich integrations	High cost, complex	Very High
Wazuh	Open source, agent-based	Steep learning curve	Free
Elastic SIEM	Scalable, performant	No AI/ML built-in	Freemium
HELIX (PoC)	Unified, AI-native, academic	Early stage	Free

Table 2.1: Comparison of Existing Security Platforms

2.4 HELIX Positioning

1. **Unified Platform:** Combines Blue Team monitoring, Red Team tools, and AI assistance in one interface.
2. **AI-Native:** Integrates machine learning anomaly detection and LLM-powered analysis from the ground up.

3. **Academic Focus:** Designed for learning, demonstration, and research with accessible architecture.

3. Project Objectives

3.1 General Objective

Develop a functional proof-of-concept for a modular cybersecurity platform that demonstrates log ingestion, anomaly detection, AI-assisted analysis, and role-based access control within a unified web interface.

3.2 Functional Objectives

- Implement user authentication with six access levels (Administrator, Blue Team Operator, Red Team Operator, Purple Team Operator, Learner, Visitor)
- Build a centralized dashboard displaying security metrics and alert summaries
- Enable manual log file upload and structured parsing
- Visualize security alerts with severity-based classification
- Apply Isolation Forest ML model to detect anomalous log patterns
- Integrate OpenAI API for natural language alert explanation
- Implement one Red Team tool (network port scanner) with controlled access
- Implement File Integrity Monitoring with SHA-256 baseline and verification

3.3 Academic Objectives

- Demonstrate full software engineering lifecycle (requirements to implementation)
- Produce complete UML diagrams (Use Case, Class, Sequence, Component, Deployment)
- Showcase multi-language integration (PHP, Python, Java, JavaScript)
- Deliver a demonstrable portfolio piece for academic evaluation

3.4 Non-Functional Objectives

- Modular architecture supporting future MVP expansion
- Clean, maintainable code following established conventions
- Secure authentication with password hashing and session management
- Responsive interface functional at standard desktop resolutions
- Comprehensive documentation for academic submission

4. User Roles and System Actors

4.1 Actor Overview

HELIX defines one abstract generalization actor (**Authenticated User**), five concrete human actors, one unauthenticated actor, and two secondary system actors:

Actor	Profile	Access Level	Type
<i>Authenticated User</i>	<i>Abstract (generalization)</i>	<i>Common auth. capabilities</i>	—
Administrator	Platform manager	Full access to all modules	Primary
Blue Team Operator	SOC analyst	Monitoring, alerts, AI, logs	Primary
Red Team Operator	Penetration tester	Offensive tools, scans, results	Primary
Purple Team Operator	Hybrid operator	Blue + Red combined view	Primary
Learner	Authenticated student	Learning materials, notes, progress	Primary
Visitor	Unauthenticated user	Public content, registration	Primary
Detection Engine	Autonomous system	ML scoring, alert generation	Secondary
AI Service	Internal system	OpenAI-powered AI responses	Secondary

Figure 4.1: HELIX PoC Actor Summary

The conceptual hierarchy is: **Visitor** (unauthenticated); **Authenticated User** → **Administrator**, **Blue Team Operator**, **Red Team Operator**, **Purple Team Operator**, **Learner**; and **Purple**

Team Operator inherits from both Blue Team Operator and Red Team Operator. Secondary system actors (Detection Engine, AI Service) interact with the system boundary externally.

4.2 Detailed Actor Descriptions

4.2.1 Actor A0 – Authenticated User (Abstract)

Role	Abstract generalization; no real user holds this role directly.
Purpose	Factors out use cases shared by all authenticated roles: Log In, Manage Profile, Use AI Assistant, Search Knowledge Base, Publish Article, Subscribe.
Inheritance	Specialized by Administrator, Blue Team Operator, Red Team Operator, Purple Team Operator, Learner.

All concrete actors inheriting from Authenticated User can perform the above use cases, unless overridden by role restrictions. The abstract actor itself never logs in—the concrete specializations do.

4.2.2 Actor A1 – Administrator

Role	Platform and user management authority
Profile	System administrator, CISO, course supervisor, developer
Technical Level	Expert
Permissions	Full access to all modules; user account management; system configuration; log management; alert oversight; AI assistant; Red Team tools; ML model management; detection rule authoring
Core Needs	Monitor platform health, manage users and roles, oversee all security operations, configure system settings, maintain detection rules

4.2.3 Actor A2 – Blue Team Operator

Role	Defensive security monitoring and incident analysis
Profile	SOC analyst, cybersecurity student, security operator
Technical Level	Intermediate
Permissions	Dashboard access; log ingestion and viewing; alert filtering and status updates; AI assistant for alert explanation; ML anomaly score consultation; report viewing
Core Needs	Identify real threats from noise, understand alert context with AI assistance, track anomaly patterns, manage incident lifecycle, generate security reports

4.2.4 Actor A3 – Red Team Operator

Role	Offensive security testing and tool execution
Profile	Penetration tester, security researcher, CTF player
Technical Level	Advanced
Permissions	Red Team tool access (port scanner, hash scanning); scan result viewing and management; OSINT queries; writeup publication; AI assistant for technique explanation
Core Needs	Execute security tools from unified interface, persist and share scan results, document techniques and findings, correlate offensive results with defensive alerts

4.2.5 Actor A4 – Purple Team Operator

Role	Cross-team coordination and detection coverage optimization (inherits Blue + Red capabilities)
Profile	Senior security engineer, team lead, hybrid operator
Technical Level	Expert
Permissions	Combined Blue + Red Team capabilities; cross-correlation queries; detection rule modification; collaborative incident management; consolidated dashboard view; findings sharing between teams
Core Needs	Visualize detection coverage, identify blind spots between offensive and defensive operations, improve detection playbooks, correlate Red Team findings with Blue Team alerts

4.2.6 Actor A5 – Learner

Role	Authenticated student and self-paced learner
Profile	Student, early-career professional, cybersecurity enthusiast
Technical Level	Beginner to Intermediate
Permissions	Access learning materials; manage personal notes; track learning progress; participate in guided exercises; AI assistant (learning mode); read-only alert and dashboard viewing
Core Needs	Document learning journey, save notes on security concepts, measure progress through platform activities, practice in safe hands-on environment, receive AI-guided explanations

4.2.7 Actor A6 – Visitor (Unauthenticated)

Role	Unauthenticated public access
Profile	General public, potential learner, recruiter, evaluator
Technical Level	Any
Permissions	Browse public content; view platform overview; register for an account; log in to access authenticated features
Core Needs	Discover platform capabilities, understand HELIX features, register to become a Learner or request elevated access

4.2.8 Actor A7 – Detection Engine (Secondary System Actor)

Type	Non-human autonomous software component (secondary actor)
Description	Triggered on log ingestion or scheduled intervals; performs ML anomaly scoring using Isolation Forest model and generates alerts based on severity thresholds
Interactions	Receives parsed log entries from PHP backend; computes anomaly scores; stores results in MySQL; triggers alert creation for HIGH/CRITICAL findings; notifies Blue Team Operator and Administrator of critical events

5. Functional and Non-Functional Requirements

5.1 Functional Requirements

5.1.1 Module 1 – Authentication and Access Control

ID	Description	Pri.	Phase
FR-001	User login with username and password verification against MySQL database.	H	PoC
FR-002	Password hashing using bcrypt before storage.	H	PoC
FR-003	PHP session management with role-based access enforcement.	H	PoC
FR-004	Nine actors: one abstract (Authenticated User), five concrete authenticated (Administrator, Blue Team Operator, Red Team Operator, Purple Team Operator, Learner), one unauthenticated (Visitor), and two secondary system actors (Detection Engine, AI Service).	H	PoC
FR-005	Session timeout and logout functionality.	M	PoC
FR-006	Protected API endpoints returning HTTP 403 for unauthorized access.	H	PoC
FR-007	Visitor can register for a Learner account.	H	PoC

Table 5.1: Functional Requirements – Authentication

5.1.2 Module 2 – Dashboard

ID	Description	Pri.	Phase
FR-010	Display total active alerts grouped by severity (Critical, High, Medium, Low).	H	PoC
FR-011	Show total ingested log count and recent log activity.	H	PoC

FR-012	Display ML anomaly score summary (average, distribution).	M	PoC
FR-013	Visual charts for alert trends over time.	M	PoC
FR-014	Role-based dashboard content (Visitor sees login page; Learner sees learning-oriented view; Purple Team Operator sees consolidated Blue+Red view).	H	PoC

Table 5.2: Functional Requirements – Dashboard

5.1.3 Module 3 – Log Ingestion

ID	Description	Pri.	Phase
FR-020	Upload log files (CSV, TXT) via web interface.	H	PoC
FR-021	Parse log entries and store in MySQL with structured fields (timestamp, source, severity, message).	H	PoC
FR-022	Validate file format and reject malformed uploads.	H	PoC
FR-023	Display ingestion status and processed entry count.	M	PoC
FR-024	Trigger ML anomaly scoring after successful ingestion.	M	PoC

Table 5.3: Functional Requirements – Log Ingestion

5.1.4 Module 4 – Alert Visualization

ID	Description	Pri.	Phase
FR-030	List all security alerts with severity, timestamp, and description.	H	PoC
FR-031	Filter alerts by severity level and date range.	H	PoC
FR-032	Color-coded severity indicators (Critical=red, High=orange, Medium=yellow, Low=green).	H	PoC
FR-033	Update alert status (Open, Investigating, Resolved) – Blue Team Operator, Purple Team Operator, and Administrator only.	M	PoC

FR-034	View alert details including associated log entry and anomaly score.	M	PoC
--------	--	---	-----

Table 5.4: Functional Requirements – Alert Visualization

5.1.5 Module 5 – ML Anomaly Detection

ID	Description	Pri.	Phase
FR-040	Python script using Isolation Forest (scikit-learn) to score log anomalies.	H	PoC
FR-041	Features: event frequency, error rate, time-of-day distribution, source IP diversity.	H	PoC
FR-042	Anomaly scores mapped to severity levels (LOW, MEDIUM, HIGH, CRITICAL).	H	PoC
FR-043	PHP backend invokes Python script and stores results in MySQL.	H	PoC
FR-044	Display anomaly scores in dashboard and alert views.	M	PoC

Table 5.5: Functional Requirements – ML Anomaly Detection

5.1.6 Module 6 – AI Assistant

ID	Description	Pri.	Phase
FR-050	Chat interface for querying about specific alerts or log entries.	H	PoC
FR-051	PHP backend forwards alert context to OpenAI API and returns explanation.	H	PoC
FR-052	AI provides remediation suggestions based on alert type and severity.	H	PoC
FR-053	Display conversation history within the session.	M	PoC
FR-054	Accessible to Administrator, Blue Team Operator, Red Team Operator, Purple Team Operator, and Learner roles.	H	PoC

Table 5.6: Functional Requirements – AI Assistant

5.1.7 Module 7 – File Integrity Monitoring

ID	Description	Pri.	Phase
FR-070	Create SHA-256 baseline hash snapshot of a target directory via Java module.	H	PoC
FR-071	Verify file integrity by comparing current hashes against stored baseline.	H	PoC
FR-072	Detect file changes: added, modified, deleted, or unchanged.	H	PoC
FR-073	Display scan results with file path, hash, and change status.	M	PoC
FR-074	View historical baselines and scan results.	M	PoC
FR-075	Accessible to Administrator, Blue Team Operator, and Purple Team Operator roles only.	H	PoC

Table 5.7: Functional Requirements – File Integrity Monitoring

5.1.8 Module 8 – Red Team Tool

ID	Description	Pri.	Phase
FR-060	Network port scanner: input target IP/hostname and scan for open ports.	H	PoC
FR-061	Display scan results with port number, protocol, and status (open/closed/filtered).	H	PoC
FR-062	Persist scan results in MySQL for future reference.	M	PoC
FR-063	Accessible to Administrator, Red Team Operator, and Purple Team Operator roles only.	H	PoC

Table 5.8: Functional Requirements – Red Team Tool

5.2 Non-Functional Requirements

5.2.1 Performance

- **NFR-001:** Dashboard loads within 2 seconds for up to 1,000 log entries.
- **NFR-002:** ML anomaly scoring completes within 5 seconds for batch of 500 entries.
- **NFR-003:** Log file upload and parsing completes within 10 seconds for files up to 5 MB.

- **NFR-004:** AI assistant response returns within 8 seconds.

5.2.2 Security

- **NFR-010:** Passwords hashed with bcrypt (cost factor ≥ 10).
- **NFR-011:** PHP sessions with secure cookie flags.
- **NFR-012:** Input validation and sanitization on all user inputs.
- **NFR-013:** SQL injection prevention via prepared statements (PDO).
- **NFR-014:** Role-based access enforced on every API endpoint.

5.2.3 Maintainability

- **NFR-020:** Modular file structure with clear separation of concerns.
- **NFR-021:** Consistent naming conventions across all languages.
- **NFR-022:** Documented API endpoints with input/output specifications.
- **NFR-023:** Configuration files separated from application logic.

5.2.4 Usability

- **NFR-030:** Interface functional at 1280x720 minimum resolution.
- **NFR-031:** Severity-based color coding consistent across all views.
- **NFR-032:** Clear error messages for failed operations.
- **NFR-033:** Intuitive navigation between modules.

5.2.5 Deployability

- **NFR-040:** Runs on XAMPP (Apache + MySQL + PHP) without additional infrastructure.
- **NFR-041:** Python ML service runs as standalone script invoked by PHP.
- **NFR-042:** Database schema provided as importable SQL file.
- **NFR-043:** Configuration via editable config files (no environment variable dependency for PoC).

Conclusion

This SRS defines the requirements for the HELIX Proof of Concept. It covers the project presentation, context and problem statement, objectives, user roles, and all functional and non-functional requirements across eight deliverable modules. The PoC scope is deliberately focused to ensure a strong, demonstrable academic deliverable within a constrained timeline, while maintaining a modular architecture that supports future MVP evolution.